RAG Mongo Demo — Enterprise Reference Documentation

A full-stack, single-tenant reference implementation of a RAG (Retrieval-Augmented Generation) pipeline for QA test-case and user-story management, built on MongoDB Atlas (Vector Search + Atlas Search/BM25), Mistral embeddings, and Groq for reranking/summarization/generation.

> **This document was produced by a full, code-level audit of the repository** (every API route, every client component, every script, every config file). Every claim is traceable to a specific file/line; anywhere a fact could not be confirmed from the code, it is stated explicitly as unknown rather than assumed. See Section 13 (Security) in particular before running this anywhere beyond a local machine.

Table of Contents

---

1. Executive Summary

> **Note on scope:** This repository is a **reference / demo implementation** of a RAG (Retrieval-Augmented Generation) pipeline for QA test-case and user-story management. It is not hardened for production multi-tenant use — see the callout at the end of this section and the full Security section later in this document.

Business Problem

QA teams accumulate large libraries of test cases and user stories (in Jira or Excel) that become difficult to search, reuse, and keep consistent with new requirements. Writing new test cases by hand for every incoming user story is slow, and keyword-only search tools miss semantically related test cases that don't share exact wording. Teams evaluating a retrieval strategy for this problem (keyword vs. semantic vs. hybrid vs. LLM-reranked) typically have no single environment to compare these approaches side by side against their own data.

Solution

This project is a full-stack sandbox that lets a team:

1. Ingest existing test cases and user stories from Excel (or Jira, via a standalone script) into structured JSON.
2. Generate vector embeddings (Mistral `mistral-embed`, 1024-dim) for that content and store it in MongoDB Atlas alongside the source documents.
3. Query that store through four distinct, independently selectable retrieval strategies — keyword (BM25), semantic (vector), hybrid (weighted BM25+vector fusion), and LLM-based reranking (Groq) — and visually compare their scoring and results.
4. Post-process results with rule-based deduplication and LLM summarization.

5. Drive an end-to-end, in-browser RAG workflow (Prompt & Schema Manager) that takes a user story, retrieves related test cases and stories, reranks and summarizes them, and asks an LLM to draft new test cases against a configurable JSON schema and prompt template.

Key Capabilities (as implemented)

Capability	Implemented as
Excel → JSON conversion (test cases & user stories)	"Convert to JSON" page + `POST /api/upload-excel`
Embedding generation & storage	"Embeddings & Store" page + `POST /api/create-embeddings(-batch)`
Query preprocessing (normalization, healthcare-domain abbreviation expansion, synonym expansion)	"Query Preprocessing" page + `POST /api/search/preprocess`
Semantic (vector) search	"Vector Search" page + `POST /api/search`
Keyword (BM25) search	"BM25 Search" page + `POST /api/search/bm25`
Weighted hybrid search	"Hybrid Search" page + `POST /api/search/hybrid`
LLM-based reranking	"Score Fusion" page + `POST /api/search/rerank` (Groq-scored, despite the UI's RRF-fusion labeling — see Sections 8/11)
Result deduplication & AI summarization	"Summarize & Dedup" page + `/api/search/deduplicate`, `/api/search/summarize`
Prompt/schema-driven test-case generation	"Prompt & Schema" page (client-side orchestration of the full pipeline + `POST /api/test-prompt`)
Environment/configuration management	"Settings" page + `GET/POST /api/env`

Target Users

- **QA / Test engineers and test architects** who want to search and reuse an existing test-case repository instead of writing duplicate tests.
- **Solution/RAG architects** evaluating which retrieval strategy (BM25 vs. vector vs. hybrid vs. reranked) best fits a given corpus, using this app as a comparison harness.
- **Developers extending the pipeline** — the scripts folder doubles as a set of standalone, runnable reference implementations for each pipeline stage.

The bundled sample data (`releases/1.21.2/stories/*.txt`, `src/data/*.json`) is healthcare-domain (hospital ward/doctor/prescription/billing user stories, `HC-xxx` IDs), and the built-in summarization prompts are explicitly written for "QA experts specializing in healthcare systems." The application itself, however, is domain-agnostic — the healthcare content is demo data, not a hard dependency.

Business Value

- Cuts duplicate test-authoring effort by surfacing semantically related existing test cases before new ones are written.
- Gives teams empirical grounds (via the side-by-side search pages) to choose a retrieval strategy rather than guessing.
- Provides a working, inspectable reference architecture (Mistral embeddings + MongoDB Atlas Search/Vector Search + Groq reranking/summarization) that can be adapted to a team's own document types beyond QA test cases.

> ⚠ **As currently implemented, this application has no authentication, authorization, rate limiting, or input sanitization on any of its 21 API endpoints.** It is suitable for local/internal demo and evaluation use, not for exposure on a shared or public network without adding an auth layer first (see Section 13).

---

2. Architecture

2.1 High-Level Architecture

The system is a **two-process, unauthenticated demo application**: a React single-page app and an Express API server, both run locally (typically together via `npm run dev`). There is no reverse proxy, gateway, or containerization layer.

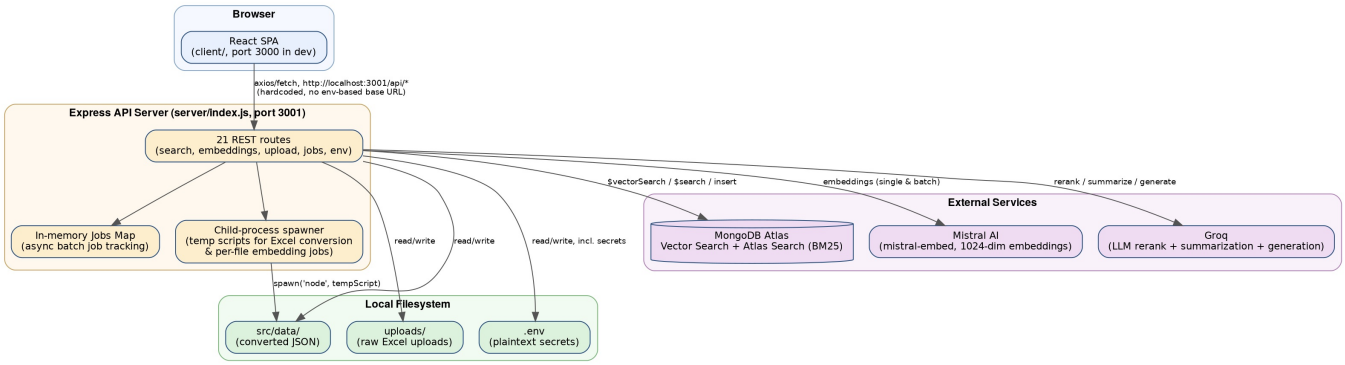

Figure 1 — High-Level Architecture

Key architectural facts: - The client never talks to MongoDB, Mistral, or Groq directly — every external call is proxied through the Express server. The server is a thin orchestration layer, not a data layer with its own models/ORM (despite `sequelize` / `mysql2` being listed dependencies — both are unused). - Two of the write-heavy routes (`/api/upload-excel` , `/api/create-embeddings`) work by **writing a JavaScript file to disk and spawning it as a child `node` process**, rather than calling functions in-process. - Async embedding jobs are tracked in an **in-memory `Map`** (`server/index.js:50`) — job state is lost on server restart, and this does not scale beyond a single server instance (the code's own comment acknowledges: "*consider using Redis for production*").

2.2 Component Architecture

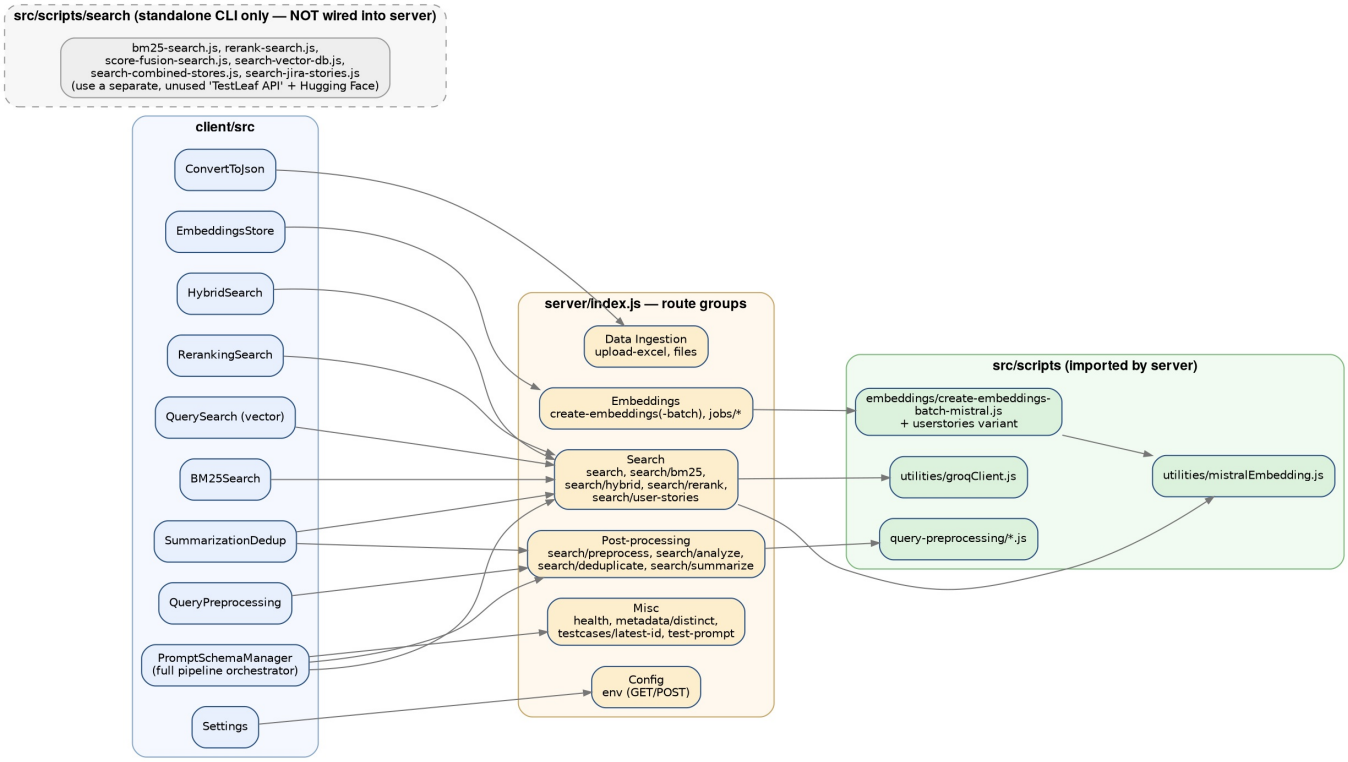

Figure 2 — Component Architecture

> **Notable finding:** `src/scripts/search/*.js` (6 files) implement a *second, independent* BM25/vector/rerank pipeline against a different embedding provider ("TestLeaf API") and Hugging Face cross-encoder reranking — none of it is called by the running server. Treat this folder as **legacy/orphaned reference code**, not part of the live system (see Section 7).

2.3 Deployment Architecture (as it exists today)

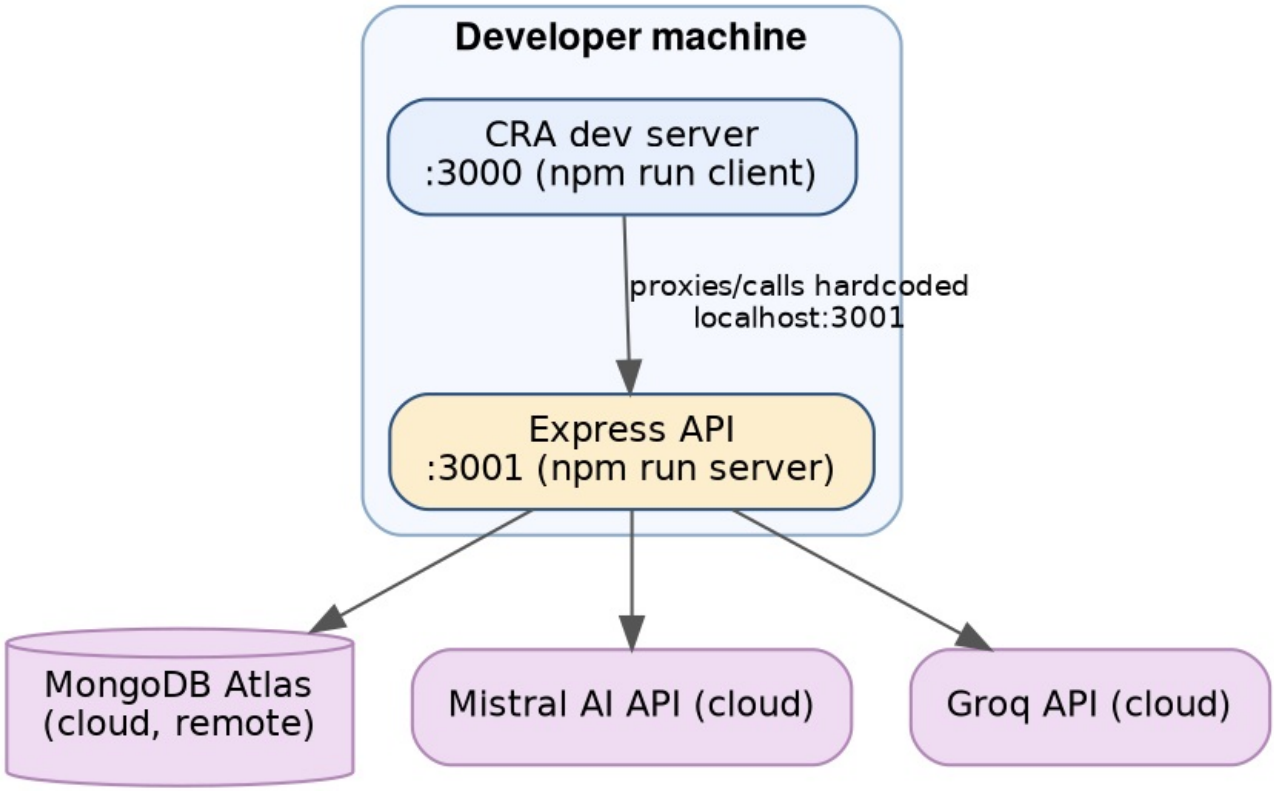

Figure 3 — Deployment Architecture

- **No production deployment path exists in this repository.** `npm run build` produces a static `client/build/` folder, but `server/index.js` never mounts it with `express.static` — only `/uploads` is served statically. There is currently no single-process way to serve the built frontend from the API server.
- The client's API base URL is hardcoded to `http://localhost:3001/api` in multiple components rather than sourced from an environment variable — this must be changed before the client and API can run on different hosts.
- Only MongoDB Atlas, Mistral, and Groq are genuinely remote/cloud dependencies; everything else is local disk state on whatever single machine runs both processes.

2.4 Sequence Diagram — Hybrid Search Request (representative core flow)

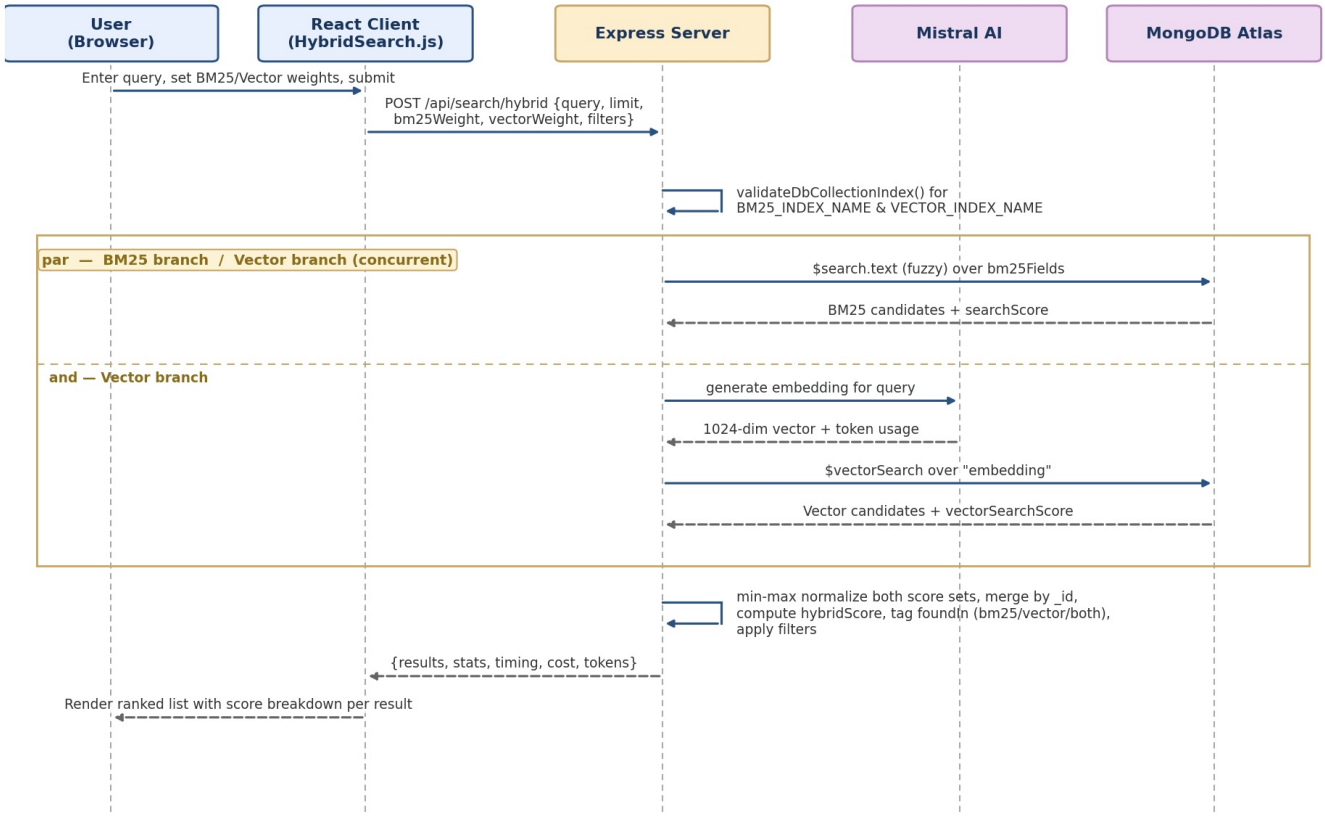

Figure 4 — Hybrid Search Request (sequence)

2.5 Data Flow — End-to-End Pipeline

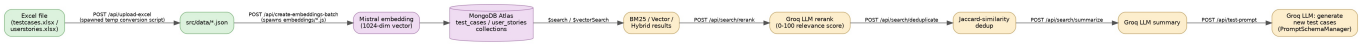

Figure 5 — End-to-End Data Pipeline

---

3. Technology Stack

3.1 Frontend

Component	Technology	Version	Notes
UI framework	React	`^19.1.1` (+ `react-dom ^19.1.1`)	Function components + hooks; no class components observed
Build tooling	Create React App (`react-scripts`)	`5.0.1`	Standard CRA scripts: `start`, `build`, `test`, `eject`
Component library	MUI (Material UI)	`@mui/material ^7.3.2`, `@mui/icons-material ^7.3.2`, `@mui/lab ^7.0.0-beta.17`, `@mui/system ^7.3.3`	Primary UI kit for all pages, drawer nav, stepper, data grid
Data grid	`@mui/x-data-grid`	`^8.13.1`	Used in `EmbeddingsStore.js` and `QuerySearch.js`
Styling engine	Emotion	`@emotion/react ^11.14.0`, `@emotion/styled ^11.14.1`	MUI's default styling engine
HTTP client	`axios`	`^1.12.2`	Used alongside raw `fetch` — usage is inconsistent across components
Notifications	`notistack`	`^3.0.2`	Toast/snackbar provider wrapping the whole app
Routing	`react-router-dom`	`^7.9.3`	**Listed but unused** — `App.js` switches views via local `useState`, not `<Routes>`
Vitals	`web-vitals`	`^2.1.4`	CRA default inclusion; no custom reporting wired to it

3.2 Backend

Component	Technology	Version	Notes
Runtime	Node.js	Not pinned — no `engines` field in either `package.json`. ES modules (`"type": "module"`) and dependencies in use are compatible with Node 18 LTS or newer,	

		but the intended minimum is undetermined from the repo.	
Web framework	Express	`^4.18.2`	Single `server/index.js` file, 21 routes, no router-file separation
File uploads	`multer`	`^1.4.5-lts.1`	Disk storage engine, uploads written to `uploads/`, **no `limits` / `fileFilter` configured**
CORS	`cors`	`^2.8.5`	Applied with no options — fully permissive
Env config	`dotenv`	`^16.4.5`	Loaded once at server startup
Excel parsing	`xlsx`	`^0.18.5`	Used both by standalone conversion scripts and the dynamically generated temp scripts in `/api/upload-excel`
Concurrent dev run	`concurrently`	`^8.2.2`	Powers `npm run dev`
Concurrency limiting	`p-limit`	`^7.1.1`	Caps parallel Mistral batch-embedding calls
MongoDB driver	`mongodb`	`^6.8.0`	Used directly — no ODM/ORM layer
Groq SDK	`groq-sdk`	`^0.5.0`	Used in `utilities/groqClient.js` and inline in `/api/test-prompt`

3.3 Database & Search

Component	Technology	Notes
Primary datastore	**MongoDB Atlas**	Two collections: `test cases` (`COLLECTION_NAME`) and `user_stories` (`USER_STORIES_COLLECTION_NAME`)
Vector database	**MongoDB Atlas Vector Search** (`$vectorSearch`)	No separate/dedicated vector DB — Atlas serves both as document store and vector index
Full-text/keyword search	**MongoDB Atlas Search** (`$search`, BM25-based, fuzzy matching)	Index existence checked by name only (`listSearchIndexes()`); field mappings in `src/config/*.json` are reference-only, never applied programmatically
Other listed DB dependencies	`mysql2 ^3.15.2`, `sequelize ^6.37.7`	**Confirmed unused** — no SQL connection code, models, or migrations exist anywhere

3.4 LLMs & AI Providers

Purpose	Provider	Model (env-configurable)	In-code fallback default	Notes
Embeddings	**Mistral AI**	`MISTRAL_EMBEDDING_MODEL` (`.env.example`: `mistral-embed`)	—	Raw REST (`axios` → `https://api.mistral.ai/v1/embeddings`), **not** the `@mistralai/mistralai` SDK despite it being a dependency
Reranking	**Groq**	`GROQ_RERANK_MODEL` (`.env.example`: `openai/gpt-oss-120b`)	`llama-3.2-3b-preview`	`utilities/groqClient.js` → `rerankDocuments()`
Summarization	**Groq**	`GROQ_SUMMARIZATION_MODEL` (`.env.example`: `openai/gpt-oss-120b`)	`llama-3.3-70b-versatile`	`utilities/groqClient.js` → `summarizeResults()`
Prompt testing / generation	**Groq**	Same as `GROQ_RERANK_MODEL`	`llama-3.2-3b-preview`	`/api/test-prompt`, used by `PromptSchemaManager.js`
Listed but unused	`openai` SDK (`^4.104.0`)	—	—	No OpenAI API calls found anywhere in the current server/client code
Legacy/orphaned only	Hugging Face (`@huggingface/inference` `^4.11.1`), a separate "TestLeaf API"	—	—	Referenced only inside the non-wired `src/scripts/search/*.js` files

3.5 Embedding Model Specification

Property	Value
Model	`mistral-embed`
Dimensions	1024 (hardcoded in `mistralEmbedding.js:getEmbeddingDimension()`, matches `numDimensions` in both `src/config/*-vector-index.json` files)
Similarity metric	Cosine
Retry policy	3x exponential backoff for single embeddings; batch calls use a longer 60s timeout

3.6 Infrastructure

Layer	Status
Containerization	**None** — no Dockerfile / docker-compose at the project level
Infrastructure-as-code	**None** found
Cloud hosting config	**None** found
Reverse proxy / gateway	**None** — client calls the API server directly at a hardcoded `http://localhost:3001`
External managed services relied on	MongoDB Atlas, Mistral AI Cloud API, Groq Cloud API

3.7 Testing

Layer	Status
Backend unit/integration tests	**None exist**
Frontend tests	**None exist** — CRA's default test scaffold is absent; `@testing-library/*` packages are listed but exercised by zero test files
Test runner configured	`client/package.json` has `"test": "react-scripts test"`; root `package.json` has **no test script at all**

3.8 CI/CD

Layer	Status
Pipeline definitions	**None** — no `.github/workflows`, Jenkinsfile, or Azure Pipelines config
Automated build/release process	**None**

3.9 Monitoring & Observability

Layer	Status
Structured logging	**None** — `console.log` / `warn` / `error` only, ~110 call sites in `server/index.js`
APM / tracing	**None** (no Sentry, Datadog, New Relic, OpenTelemetry)
Metrics/dashboards	**None**
Health check	`GET /api/health` exists but is a static liveness stub — does not check MongoDB/Mistral/Groq connectivity

---

4. Features

4.1 Convert to JSON

- **Purpose:** Turn raw Excel exports (test cases or user stories) into the structured JSON format the rest of the pipeline expects.
- **Workflow:** Select data type (Test Cases / User Stories) → pick file and sheet name → Convert → view conversion log/result → converted file becomes selectable on the Embeddings & Store page.
- **Implementation overview:** Multer receives the upload into `uploads/`. The server builds a column-mapping script as a string, writes it to disk as a temp `.js` file, and spawns it as a child `node` process (30s timeout) to do the `xlsx` parsing and field mapping, then deletes the temp script and the original upload.
- **Files involved:** `client/src/components/data/ConvertToJson.js`; `server/index.js` `POST /api/upload-excel`; output lands in `src/data/`.
- **Advantages:** Flexible column-header aliasing; no separate conversion microservice needed.
- **Limitations:** The same mapping logic is duplicated independently in `src/scripts/data-conversion/excel-to-userstories.js`; the 30-second child-process timeout could fail on very large workbooks.
- **Example:** Upload `userstories.xlsx`, `dataType=userstories`, `sheetName=stories` → produces `src/data/stories-<timestamp>.json`.

4.2 Embeddings & Store

- **Purpose:** Generate vector embeddings for converted JSON and persist them in MongoDB so the content becomes searchable.
- **Workflow:** Select one or more files from the `src/data` grid → start embedding job → live progress polling → completion summary.
- **Implementation overview:** `POST /api/create-embeddings-batch` validates the DB/collection exist, creates an in-memory job entry, and spawns the appropriate batch script with `EMBEDDING_INPUT_FILES` set; batches 50 records per Mistral call (up to 3 concurrent via `p-limit`), writes to Mongo in batches of 100 with retry/backoff. Client polls `GET /api/jobs/:jobId` every 2 seconds.
- **Files involved:** `client/src/components/data/EmbeddingsStore.js`; `server/index.js` (`/api/files`, `/api/jobs/*`, `/api/create-embeddings`, `/api/create-embeddings-batch`); `src/scripts/embeddings/create-embeddings-batch-mistral.js` (+ userstories variant); `utilities/mistralEmbedding.js`.
- **Advantages:** The batch route is materially faster than the older per-document `/api/create-embeddings` path; retry/backoff

improves resilience.
- **Limitations:** Job state is in-memory only — lost on server restart, not shareable across instances; the client chooses test-case vs. user-story script by filename string-matching; the component still contains a leftover "Debug Info" panel.
- **Example:** Selecting `stories-1785161723926.json` runs the user-stories batch script and inserts documents with a 1024-dim `embedding` field into the `user_stories` collection.

4.3 Query Preprocessing

- **Purpose:** Normalize and expand a raw query (case/whitespace, healthcare-domain abbreviations, synonyms) before it is used for retrieval, with a visual breakdown of each transformation.
- **Workflow:** Enter a query → Preprocess → inspect Pipeline Steps / Abbreviations / Synonym Variations tabs → optionally run a generated variant through search.
- **Implementation overview:** `POST /api/search/preprocess` dynamically imports `queryPreprocessor.js`, which chains `normalize()` → `expandAbbreviations()` (healthcare dictionary in `dictionaries.js`) → `expandSynonyms()`, and re-attaches any extracted test-case IDs (`TC_<n>` pattern).
- **Files involved:** `client/src/components/processing/QueryPreprocessing.js`; `server/index.js` (`/api/search/preprocess`, `/api/search/analyze`); `src/scripts/query-preprocessing/*.js`.
- **Advantages:** Pure logic, no external API calls — fast and deterministic; ambiguous abbreviations (`ip`/`op`/`er`/`tc`) are flagged rather than silently expanded.
- **Limitations:** This page's own "Search Results" tab hardcodes `searchType` to vector search with no UI control exposed.
- **Example:** `"pt bp check in OP"` → normalized and abbreviation-expanded toward `"patient blood pressure check in outpatient"`, plus several synonym variants.

4.4 Vector Search

- **Purpose:** Semantic retrieval of test cases via MongoDB Atlas `$vectorSearch`.
- **Workflow:** Enter query + optional module/priority/risk/type filters → Search → DataGrid of results with similarity % and an expandable detail view.
- **Implementation overview:** `POST /api/search` embeds the query with Mistral, runs `$vectorSearch` (`numCandidates = max(100, limit×10)`) against `VECTOR_INDEX_NAME`, applies metadata filters as a **post-search** `$match`, returns cost/token usage.
- **Files involved:** `client/src/components/search/QuerySearch.js`; `server/index.js` `POST /api/search`; `utilities/mistralEmbedding.js`.
- **Advantages:** Surfaces conceptually related test cases without shared keywords; filter dropdown options are sourced live from stored data (`GET /api/metadata/distinct`).
- **Limitations:** Filters are applied after the vector search rather than natively inside the `$vectorSearch` stage.
- **Example:** Query "patient queue not refreshing" can surface a real-time-dashboard test case even without the word "queue" appearing in it.

4.5 BM25 Search

- **Purpose:** Pure keyword/full-text retrieval using Atlas Search's BM25-based scoring.
- **Workflow:** Enter query + filters → Search → results with a raw (unbounded) score and a color-coded confidence band.
- **Implementation overview:** `POST /api/search/bm25` validates the index and non-empty collection, runs `$search.text` with fuzzy matching (`maxEdits:1, prefixLength:2`), adds `searchScore`, applies filters via `$match`.
- **Files involved:** `client/src/components/search/BM25Search.js`; `server/index.js` `POST /api/search/bm25`.
- **Advantages:** Reliable for exact/near-exact keyword and ID matches (e.g. `TC_045`).
- **Limitations:** The per-field weighting present in the standalone `src/scripts/search/bm25-search.js` is **not** used here — the live route treats all requested fields equally.
- **Example:** Query `"TC_045"` reliably surfaces the exact test case by ID.

4.6 Hybrid Search

- **Purpose:** Blend BM25 and vector search to balance keyword precision with semantic recall.
- **Workflow:** Enter query, adjust linked BM25/Vector weight sliders (sum to 100%) or choose a preset → Search → per-result score breakdown with a `foundIn` badge.
- **Implementation overview:** `POST /api/search/hybrid` runs BM25 and vector search, independently min-max normalizes each score set, merges by `_id` into `hybridScore = bm25Norm×bm25Weight + vectorNorm×vectorWeight`.
- **Files involved:** `client/src/components/search/HybridSearch.js`; `server/index.js` `POST /api/search/hybrid`.
- **Advantages:** A single tunable weight makes the retrieval trade-off directly visible.
- **Limitations:** The BM25 and vector branches are awaited sequentially rather than dispatched concurrently.
- **Example:** A 70/30 keyword-heavy preset ranks an exact-phrase match above a loosely related semantic match.

4.7 Score Fusion / LLM Reranking

- **Purpose:** Use an LLM (Groq) to relevance-score and reorder a candidate pool, as an alternative to purely numeric fusion.
- **Workflow:** Enter query, set `rerankTopK` and final `limit` → Search → "After Reranking," "Before Reranking," and rank-comparison tabs.
- **Implementation overview:** `POST /api/search/rerank` fetches ~50 BM25 + ~50 vector candidates, dedupes by `id`, sends the

pool to Groq (`rerankDocuments`), which scores each 0–100; results sorted by normalized `rerankScore` .
- **Files involved:** `client/src/components/search/RerankingSearch.js` ; `server/index.js` `POST /api/search/rerank` and `handleGroqOnlyReranking` helper; `utilities/groqClient.js` .
- **Advantages:** Can capture relevance judgments lexical/vector scoring misses; falls back gracefully to original order if the LLM call fails.
- **Limitations — confirmed discrepancy:** The UI is labeled "Score Fusion" and sends `fusionMethod:'rrf'` / `useGroqOnly:true` , but the server has **only one code path** (Groq-only reranking) and silently ignores both flags. There is also no distinct pre-rerank list returned by the API, so the "Before Reranking" tab shows the **same** array as "After Reranking."
- **Example:** `rerankTopK=50, limit=10` → 50 fused candidates scored by Groq, top 10 returned by LLM relevance.

4.8 Summarize & Dedup

- **Purpose:** Reduce a result set to unique test cases and generate a natural-language summary of what they cover.
- **Workflow:** Search → Deduplicate (adjustable threshold) → Summarize (concise/detailed) → tabbed results.
- **Implementation overview:** `POST /api/search/deduplicate` computes pairwise Jaccard similarity over lowercased **titles only**; `POST /api/search/summarize` builds a structured text block and asks Groq for a concise or detailed (healthcare-QA-framed) summary.
- **Files involved:** `client/src/components/processing/SummarizationDedup.js` ; `server/index.js` `POST /api/search/deduplicate` , `POST /api/search/summarize` ; `utilities/groqClient.js` .
- **Advantages:** Dedup needs no LLM call — fast and free; detailed summarization prompt is domain-tuned.
- **Limitations:** Title-only comparison can cause false positives/negatives; reported cost is hardcoded to `$0` .
- **Example:** 20 raw hybrid results → threshold 0.85 → 3 flagged duplicates → 17 unique results summarized.

4.9 Prompt & Schema Manager

- **Purpose:** Configure the JSON schema and prompt template used for AI-assisted test-case generation, and run the full RAG pipeline end-to-end starting from a user story.
- **Workflow:** Edit JSON Schema tab and Prompt Template tab (ICEPOT structure) → Test & Preview tab runs the pipeline → Metrics Evaluation tab scores the output.
- **Implementation overview:** Client-side `handleLlmRagTest` chains preprocess → hybrid search (test cases) → hybrid search (user stories) → rerank → deduplicate → summarize → fetch next test-case ID → generate via `POST /api/test-prompt` → parse/repair/validate JSON output. Schema/template persisted **only to** `localStorage` .
- **Files involved:** `client/src/components/processing/PromptSchemaManager.js` (~3,115 lines — the largest component in the app); most search/post-processing routes plus `GET /api/testcases/latest-id` and `POST /api/test-prompt` .
- **Advantages:** Demonstrates the entire pipeline as one observable workflow; auto-generates the next sequential `TC_<n>` ID; CSV export.
- **Limitations:** The "Metrics Evaluation" tab calls `POST http://localhost:8000/eval` — a service **not part of this repository**. Schema/template config is per-browser only. A dead, commented-out `handleTest` function remains in the file.
- **Example:** Feed in the HC-287 story → pipeline retrieves related test cases/stories → Groq drafts new candidate test cases, numbered from the next available `TC_xxx` .

4.10 Settings

- **Purpose:** View and edit the application's environment variables from the browser instead of hand-editing `.env` .
- **Workflow:** Open Settings → view masked sensitive fields with a show/hide toggle → edit → Save → banner warns a restart is required.
- **Implementation overview:** `GET /api/env` reads and parses `.env` into flat JSON; `POST /api/env` **overwrites** `.env` from the submitted object — no schema/type validation.
- **Files involved:** `client/src/components/settings/Settings.js` ; `server/index.js` `GET` / `POST /api/env` .
- **Advantages:** One place to see/update all 12 known config fields.
- **Limitations — significant, not cosmetic:** Both endpoints are **completely unauthenticated**. Any client able to reach port 3001 can read every secret in `.env` or overwrite it arbitrarily. The "sensitive" masking is a client-side display convenience only. Full detail in Section 13.
- **Example:** Rotating `GROQ_API_KEY` : paste the new value, Save, then manually restart `node server/index.js` .

---

5. Installation

5.1 Prerequisites

Requirement	Notes
Node.js	No version pinned anywhere in this repo. Modern LTS recommended in practice, not a documented hard requirement.
npm	Ships with Node; only `package-lock.json` files are present (no Yarn/pnpm).
MongoDB Atlas account	A cluster with **Atlas Search** and **Atlas Vector Search** enabled.
Mistral AI account	For an API key used to generate embeddings (`mistral-embed`).

Groq account	For an API key used for reranking, summarization, and generation.
Docker	**Not required and not supported** — no Dockerfile/compose file exists.

5.2 Install Dependencies

```
npm install
```

This installs root dependencies and, via `postinstall`, automatically runs `cd client && npm install` too.

5.3 Configure Environment Variables

```
cp .env.example .env
```

Variables defined in `.env.example`

Variable	Purpose	Required for
`MONGODB_URI`	Atlas connection string	All DB-backed routes
`DB_NAME`	Database name	All DB-backed routes
`COLLECTION_NAME`	Test-case collection	Test-case embeddings/search
`VECTOR_INDEX_NAME`	Vector Search index name, test cases	Vector/hybrid/rerank search
`BM25_INDEX_NAME`	Atlas Search index name, test cases	BM25/hybrid/rerank search
`USER_STORIES_COLLECTION_NAME`	User-stories collection	User-story embeddings/search
`USER_STORIES_VECTOR_INDEX_NAME`	Vector Search index name, user stories	`/api/search/user-stories`
`MISTRAL_API_KEY`	Mistral AI API key	All embedding generation
`MISTRAL_EMBEDDING_MODEL`	Embedding model name (template default: `mistral-embed`)	Embedding generation
`GROQ_API_KEY`	Groq API key	Reranking, summarization, test-prompt/generation
`GROQ_RERANK_MODEL`	Groq model for reranking (template: `openai/gpt-oss-120b`; in-code fallback: `llama-3.2-3b-preview`)	`/api/search/rerank`, `/api/test-prompt`
`GROQ_SUMMARIZATION_MODEL`	Groq model for summarization (template: `openai/gpt-oss-120b`; in-code fallback: `llama-3.3-70b-versatile`)	`/api/search/summarize`

Variables used in code but absent from `.env.example` (documentation gap found)

Variable	Where used	Behavior if unset
`USER_STORIES_BM25_INDEX_NAME`	`server/index.js:2431`, `/api/search/user-stories`	Route falls back to **vector-only** search for user stories
`PORT`	`server/index.js:25`	Defaults to `3001`
`JIRA_BASE_URL`, `JIRA_EMAIL`, `JIRA_API_TOKEN`, `JIRA_PROJECT_KEY`	Standalone `fetch-jira-stories.js` CLI script only	Needed only if pulling real stories from Jira manually

> The `src/scripts/search/*.js` legacy scripts additionally reference `TESTLEAF_API_BASE`, `USER_EMAIL`, `AUTH_TOKEN`, `HUGGINGFACE_API_KEY` / `HF_TOKEN` — not needed to run the live application.

5.4 Database & Search Index Setup

This app does **not** create Atlas Search/Vector Search indexes programmatically. Steps:

1. Create/confirm the two collections named in your `.env`.
2. In Atlas, create a **Vector Search** index on each collection using `src/config/testcases-vector-index.json` / `user-stories-vector-index.json` as field definitions, named to match `VECTOR_INDEX_NAME` / `USER_STORIES_VECTOR_INDEX_NAME`.
3. Create an **Atlas Search** index on the test-case collection using `src/config/testcases-bm25-index.json`, named to match `BM25_INDEX_NAME`.
4. Optionally, create a matching index on user stories using `src/config/user-stories-bm25-index.json`, named to match `USER_STORIES_BM25_INDEX_NAME` — otherwise user-story search runs vector-only.
5. ⚠ Note: `user-stories-vector-index.json` defines filter paths like `jiraMetadata.projectKey` / `status.name`, but Excel-based ingestion stores a flat `status:{name,category}` with no `jiraMetadata` wrapper — adjust filter paths if using Excel-sourced stories.

5.5 API Keys

Key	Where to get it

Mistral API key	https://console.mistral.ai
Groq API key	https://console.groq.com

The server checks for the literal placeholder `your_groq_api_key_here` and throws a descriptive error if still present — Mistral has no equivalent check.

5.6 Verification

```
npm run dev
```

1. `curl http://localhost:3001/api/health` → expect `{"status":"OK",...}`.
2. `curl http://localhost:3001/api/metadata/distinct` → expect a `metadata` object.
3. Open `http://localhost:3000` → "Convert to JSON" loads by default.
4. Upload a small Excel file → confirm it appears in the Embeddings & Store file list.
5. Run an embedding batch job, then try Vector Search and BM25 Search to confirm both paths work end-to-end.

---

6. Running the Project

6.1 Root `package.json` Scripts

Script	Command	Purpose
`postinstall`	`cd client && npm install`	Runs automatically after `npm install`
`dev`	`concurrently "npm run server" "npm run client"`	Runs API + client together
`server`	`node server/index.js`	Starts the Express API alone (`PORT`, default `3001`)
`client`	`cd client && npm start`	Starts the CRA dev server alone (port `3000`)
`build`	`cd client && npm run build`	Produces a static production bundle at `client/build/`

No `start`, `test`, or `lint` script exists at the root.

6.2 Development

```
npm run dev
```

API on `http://localhost:3001`, React dev server on `http://localhost:3000` (no proxy field configured; relies on the server's permissive CORS).

6.3 Build

```
npm run build
```

Produces `client/build/` — **not automatically served by anything**; requires a separate static file server.

6.4 Production

No dedicated production path exists. At minimum: build the client, serve it separately, run `node server/index.js` under a process manager, fix the hardcoded API base URL, point at production Mongo credentials, and add a reverse proxy/TLS if exposed beyond localhost.

6.5 Docker

Not supported — no Dockerfile or docker-compose file exists anywhere in this repository.

6.6 Debug

No debug tooling ships with this repo (no `.vscode/launch.json`). Use `node --inspect server/index.js` manually and attach a debugger; the client can be debugged with standard React/browser DevTools against the CRA dev server.

---

7. Project Structure

```
rag-mongo-demo-v9/
```

```
├── .env                          # Local secrets (gitignored)
├── .env.example                  # Template for .env
├── .gitignore
├── package.json                  # Root scripts + backend dependencies
├── package-lock.json
│
├── client/                       # React frontend (Create React App)
│   ├── package.json
│   ├── public/
│   ├── build/                    # Generated bundle (gitignored; NOT served by server/index.js)
│   └── src/
│       ├── index.js
│       ├── App.js                # Theme, drawer nav, view-switching between the 10 pages
│       └── components/
│           ├── data/
│           │   ├── ConvertToJson.js
│           │   └── EmbeddingsStore.js
│           ├── processing/
│           │   ├── QueryPreprocessing.js
│           │   ├── SummarizationDedup.js
│           │   └── PromptSchemaManager.js    # ~3,115 lines, full pipeline orchestrator
│           ├── search/
│           │   ├── QuerySearch.js
│           │   ├── BM25Search.js
│           │   ├── HybridSearch.js
│           │   └── RerankingSearch.js
│           └── settings/
│               └── Settings.js
│
├── server/
│   └── index.js                  # Entire backend: 21 routes, ~2,749 lines, single file
│
├── src/
│   ├── config/                   # Reference-only Atlas index definitions (not auto-applied)
│   │   ├── testcases-vector-index.json
│   │   ├── testcases-bm25-index.json
│   │   ├── user-stories-vector-index.json
│   │   └── user-stories-bm25-index.json
│   ├── data/                     # Source Excel + generated JSON (runtime working directory)
│   └── scripts/
│       ├── data-conversion/      # Standalone CLI converters
│       ├── embeddings/           # ACTIVE — invoked by the server
│       ├── query-preprocessing/  # ACTIVE — imported by the server
│       ├── search/                # ⚠ ORPHANED — not imported by server or client
│       └── utilities/            # ACTIVE — mistralEmbedding.js, groqClient.js, delete-all-documents.js
│
├── releases/
│   └── 1.21.2/stories/           # Sample healthcare (HC-xxx) demo user-story .txt files (not ingested by code)
│
└── uploads/                      # Runtime scratch space for Multer uploads (not gitignored, usually empty)
```

7.1 Folder Responsibility Summary

Folder	Responsibility	Status
`client/`	Entire frontend — 10 pages, no routing library	Active
`server/`	Entire backend — one file, 21 routes	Active
`src/config/`	Atlas index schema references	Reference-only, not executed
`src/data/`	Runtime working directory	Active, runtime-generated
`src/scripts/data-conversion/`, `embeddings/`, `query-preprocessing/`, `utilities/`	Reusable logic + CLI tools	Mixed — some imported live, others standalone CLI
`src/scripts/search/`	A second, independent search/rerank implementation	**Orphaned** — different embedding provider, not called by anything live
`releases/`	Sample demo user-story text files	Static reference content
`uploads/`	Multer's upload destination	Transient — deleted immediately after conversion

7.2 Files Worth Flagging Individually

- **`server/index.js`** (~2,749 lines) — the entire backend in one file: routing, MongoDB access, child-process spawning, all business logic.
- **`client/src/components/processing/PromptSchemaManager.js`** (~3,115 lines) — the largest source file in the repository.
- **`src/scripts/utilities/delete-all-documents.js`** — a 27-line script that runs `collection.deleteMany({})` **unconditionally, with no confirmation prompt**.

- `.env` — correctly `.gitignore`-excluded, but its contents are fully exposed and rewritable at the application level via `GET` / `POST` `/api/env` regardless (Section 13).

---

8. API Documentation

Base URL: `http://localhost:3001/api` (hardcoded on the client in most components).

Authentication: ⚠ **None on any route.** No API key, session, or token is checked anywhere. Every route below is reachable by any client that can open a TCP connection to the server's port.

Common headers: `Content-Type: application/json` for all routes except `POST /api/upload-excel` (`multipart/form-data`).

Common error shape: Most failures return `500` with `{ "error": "...", "details": "..." }`; input-validation failures return `400`. There is no centralized error-handling middleware.

8.1 Health & Jobs

`GET /api/health` — Liveness check only (does not verify Mongo/Mistral/Groq). Response: `{ "status": "OK", "message": "Server is running" }`.

`GET /api/jobs/active` — Lists in-progress async jobs. Response: `{ "jobs": [...] }`.

`GET /api/jobs/:jobId` — Poll a specific job's status. `404` if pruned (>1hr old) or never existed.

8.2 Metadata & Files

`GET /api/metadata/distinct` — Distinct `module` / `priority` / `risk` / `automationManual` values for filter dropdowns. Response: `{ "success": true, "metadata": {...} }`.

`GET /api/files` — Lists `*.json` files in `src/data/`.

8.3 Data Ingestion & Embeddings

`POST /api/upload-excel` (multipart) — Body: `file`, `dataType` (`testcases` | `userstories`), `sheetName`. Converts via a dynamically generated, spawned Node script. `400` if no file; `500` on conversion failure (incl. 30s timeout).

`POST /api/create-embeddings` — Legacy per-document embedding path. Body: `{ "files": [...] }`. Fire-and-forget; poll job status.

`POST /api/create-embeddings-batch` — Preferred batch path. Body: `{ "files": [...], "scriptName": "...", "jobType": "..." }`.

8.4 Configuration

`GET /api/env` ⚠ — Returns full `.env` contents as JSON, **no auth, no filtering**. Exposes `MONGODB_URI`, `MISTRAL_API_KEY`, `GROQ_API_KEY` in plaintext.

`POST /api/env` ⚠ — Overwrites `.env` with submitted key/values, **no auth, no validation**. Takes effect only after a manual restart.

8.5 Query Preprocessing

`POST /api/search/preprocess` — Body: `{ "query": "...", "options": {...} }`. Runs normalize → abbreviation-expand → synonym-expand.

`POST /api/search/analyze` — Dry-run diagnostics of what preprocessing would do.

8.6 Post-Processing

`POST /api/search/deduplicate` — Body: `{ "results": [...], "threshold": 0.85 }`. Jaccard-similarity title dedup.

`POST /api/search/summarize` — Body: `{ "results": [...], "summaryType": "concise"|"detailed", "query": "..." }`. Groq-generated summary; cost always reported as `0`.

`POST /api/test-prompt` — Body: `{ "prompt": "...", "temperature": 0.2, "maxTokens": 4000 }`. Generic Groq chat-completion pass-through with JSON-repair fallback parsing.

8.7 Search

`POST /api/search` (Vector) — Body: `{ "query": "...", "limit": 5, "filters": {} }`. Embeds query via Mistral, runs `$vectorSearch`.

`POST /api/search/bm25` — Body: `{ "query": "...", "limit": 10, "filters": {}, "fields": [...] }`. Runs `$search.text` with fuzzy

matching. `score` is raw/unbounded.

`POST /api/search/hybrid` — Body: `{ "query": "...", "limit": 10, "bm25Weight": 0.5, "vectorWeight": 0.5 }`. Min-max normalized weighted fusion; per-result `foundIn` tag.

`POST /api/search/rerank` — Body: `{ "query": "...", "limit": 10, "rerankTopK": 50 }`. Groq-only LLM reranking (client's `fusionMethod` / `useGroqOnly` flags are ignored server-side).

`GET /api/testcases/latest-id` — Scans all `id` fields for max `TC_<n>`, returns next available ID. Full collection scan (Section 14.2).

`POST /api/search/user-stories` — Body: `{ "query": "...", "limit": 10, "bm25Weight": 0.4, "vectorWeight": 0.6 }`. Hybrid search over `user_stories`, with rule-based `impactReason` / `regressionRisk` added per result. Falls back to vector-only if `USER_STORIES_BM25_INDEX_NAME` is unset/missing.

---

9. UI Walkthrough

9.1 Global Layout & Navigation

The frontend is a single-page app with **no routing** (`react-router-dom` is installed but unused) — navigation is pure React state in `App.js`. There is no deep-linking; refreshing always returns to "Convert to JSON."

Persistent chrome: a top App Bar (drawer toggle, title, dark/light switch persisted to `localStorage`), a breadcrumb strip, and a collapsible left Drawer (280px ↔ 72px, becomes a temporary overlay under 768px).

10 navigation items: Convert to JSON (default), Embeddings & Store, Query Preprocessing, Vector Search, BM25 Search, Hybrid Search, Score Fusion, Summarize & Dedup, Prompt & Schema, Settings.

9.2 Page-by-Page Summary

- **Convert to JSON** — 3-step Stepper (Select File → Configure → Convert) with a conversion log.
- **Embeddings & Store** — file-picker DataGrid + live job progress; includes a leftover "Debug Info" panel.
- **Query Preprocessing** — query input, 4 tabs (Pipeline Steps / Abbreviations / Synonym Variations / Search Results); search type fixed to vector.
- **Vector Search** — query + filters, DataGrid results with similarity % and expandable detail.
- **BM25 Search** — query + filters, raw BM25 score with color-coded confidence band.
- **Hybrid Search** — query + linked weight sliders/presets, per-result score breakdown and `foundIn` chip.
- **Score Fusion** — query + `rerankTopK` / `limit`; "Before"/"After" tabs currently render identical data (Section 4.7).
- **Summarize & Dedup** — search → dedup → summarize pipeline with 3 result tabs.
- **Prompt & Schema** — 4 tabs (JSON Schema, Prompt Template, Test & Preview, Metrics Evaluation); the last tab requires an external service on port 8000 not included in this repo.
- **Settings** — masked sensitive fields, Save banner warning a restart is required.

9.3 Typical User Journey

Convert → Embed → compare across Vector/BM25/Hybrid/Score Fusion → Summarize & Dedup → Prompt & Schema (generate + export CSV) → Settings (rotate keys as needed).

9.4 Responsive Behavior

Single breakpoint (`768px`) switches the drawer between permanent and temporary/overlay modes; no other custom responsive layouts beyond MUI's own component defaults.

---

10. Data Pipeline

10.1 Stages Overview

```
Excel Upload → Conversion (Excel→JSON) → Embedding Generation → MongoDB Storage
    → Retrieval (BM25 / Vector / Hybrid) → Reranking (optional) → Deduplication (optional)
    → Summarization (optional) → LLM Generation (optional, produces new test cases)
```

10.2 Upload

Received by Multer into `uploads/<timestamp>-<originalname>`, no content validation beyond what `xlsx` naturally rejects.

10.3 Conversion

Test cases — flat mapping: `module, testCaseId→id, preRequisites, title, description, steps, expectedResults, automationManual, priority, createdBy, createdDate, lastModifiedDate, linkedStories, risk, version, type`. Missing columns become empty strings — no required-field enforcement.

User stories — richer mapping with many header aliases and structural transforms (`status:{name,category}`, `priority:{name,id}`, comma-split arrays, Excel serial-date parsing). Rows are dropped only if `summary` is missing or still `"Untitled User Story"`.

⚠ This logic is duplicated independently between the server's inline conversion script and `src/scripts/data-conversion/excel-to-userstories.js`.

10.4 Embedding Generation

Only 6 of ~15 test-case fields are embedded (`ID, Module, Title, Description, Steps, Expected Result`) — fields like `priority`, `risk`, `automationManual` are stored but not embedded, so they cannot influence vector-search relevance. Batches of 50, up to 3 concurrent Mistral calls, 3 retries with exponential backoff.

10.5 Storage (confirmed exact document shape)

```
{
  ...testcase,
  embedding: [ /* 1024 floats */ ],
  embeddingMetadata: {
    model: "mistral-embed",
    cost: 0.0000012,
    tokens: 42,
    apiSource: "mistral-batch",   // legacy per-document path instead tags "testleaf" — inconsistent labeling, both are actually
Mistral
    batchNumber: 3,
    createdAt: "..."
  }
}
```

Inserted in batches of 100 via `insertMany`, `p-limit`-controlled with its own retry logic.

10.6 Retrieval

Stored documents are queried via `$search` (BM25) and/or `$vectorSearch`, each attaching a score field. `PromptSchemaManager.js` explicitly strips the `embedding` field before calling `/api/search/deduplicate` to avoid HTTP 413 errors.

10.7 Reranking (optional)

Groq scores each candidate 0–100, normalized to `rerankScore`. This stage discards the original BM25/vector scores entirely — not recoverable from the response.

10.8 Deduplication (optional)

Jaccard similarity over lowercased `title` strings only, `O(n²)`. Applied automatically only inside `PromptSchemaManager.js`, and only above 5 results.

10.9 Summarization (optional)

Serializes every available field per result into a prompt sent to Groq — no truncation/token-budget guard found before this prompt is sent.

10.10 Generation (loop-closing stage, Prompt & Schema Manager only)

Produces new candidate test cases via `/api/test-prompt`, auto-numbered from `GET /api/testcases/latest-id`. **Not automatically written back into MongoDB or `src/data/`** — CSV export is the only persistence path; re-ingestion requires manual re-upload through Stage 1.

---

11. AI Architecture

11.1 Two-Provider Model

Provider	Role

Mistral AI (`mistral-embed`)	Embeddings only — no generative calls
Groq	All generative/inference tasks — reranking, summarization, prompt completion; treated as free-tier in code (`cost` hardcoded to `0` in two routes)

No shared LLM-gateway abstraction exists between the two provider wrapper modules.

11.2 Embedding Pipeline Architecture

`mistralEmbedding.js` calls Mistral's REST endpoint directly via `axios`, not the listed SDK. Two entry points: `generateEmbedding()` (single, 3x retry) and `generateBatchEmbeddings()` (array, 60s timeout). Only 6 fixed fields are embedded per test case (Section 10.4). Dimensionality (1024) is hardcoded in three separate places with no single source of truth.

11.3 Retrieval Fusion: Two Distinct Architectures

	Hybrid Search	**Score Fusion / "Rerank"**
Mechanism	Deterministic min-max normalize + weighted sum	LLM (Groq) scores each candidate 0–100
Cost/latency	Cheap, fast	Expensive, slower — full Groq completion call
Determinism	Fully deterministic	**Non-deterministic**
Failure behavior	Fails the whole request	Falls back to original order gracefully
Client-requested params honored?	Yes (`bm25Weight` / `vectorWeight`)	`fusionMethod` / `useGroqOnly` sent but **ignored server-side**

"Score Fusion" is a UI naming carryover — the live implementation is LLM reranking, not RRF or numeric fusion of the two channels' scores.

11.4 LLM Integration Layer (`utilities/groqClient.js`)

Exposes `rerankDocuments()`, `summarizeResults()`, and an unused `generateAnswer()` (dead code). All Groq calls are **non-streaming** — the full completion is awaited before any response reaches the client, even for the longest generation call (`maxTokens` up to 10,000 from the client). Every generative route implements its own ad hoc JSON-repair heuristics (strip fences, balance braces) rather than a shared contract.

11.5 Prompt Manager

Lives in `PromptSchemaManager.js`'s "Prompt Template" tab, ICEPOT-structured (Instruction/Context/Examples/Persona/Output/Tone). **Persistence: `localStorage` only** — no server-side store, versioning, or audit trail.

11.6 Schema Manager

Defines the target output shape, used both to instruct the LLM and to validate/repair its JSON response. No formal JSON Schema validation library (`ajv` / `zod`) is used — validation is the same ad hoc repair logic, not structurally aware. Also `localStorage`-only.

11.7 Who Orchestrates the RAG Pipeline

The multi-step orchestration (preprocess → search → rerank → dedup → summarize → generate) lives **entirely in the React client**, not the server. The server exposes only atomic endpoints; the client sequences 8+ HTTP calls. This orchestration is not reusable outside the browser.

11.8 AI-Layer Risks

- No prompt-injection consideration — retrieved content is concatenated directly into prompts with no sanitization.
- `maxTokens` inconsistency: client requests up to 10,000, one server-side cap is 8,000 — worth verifying which wins.
- No non-determinism handling/seed-lock for reranking or generation.
- No prompt/schema governance across users, since both are `localStorage`-only.

---

12. Configuration

Configuration spans four independent layers with no single source of truth: `.env` (server/scripts), `src/config/*.json` (disconnected reference files), hardcoded client constants, and browser `localStorage`.

12.1 Layer 1 — `.env`

See Section 5.3 for the full variable table, including the confirmed documentation gaps (`USER_STORIES_BM25_INDEX_NAME`, `PORT` — used in code but absent from `.env.example` and the Settings UI's known-field list).

> ⚠ Any variable not among the 12 fields the Settings UI knows about (e.g. `USER_STORIES_BM25_INDEX_NAME`, `PORT`) would be **silently dropped** from `.env` the next time someone saves via Settings, since that save regenerates the whole file from only the fields the UI tracks.

12.2 Layer 2 — Atlas Index Definitions (`src/config/*.json`)

Not consumed by any code path. Editing these files has no effect on a running system unless separately re-applied to Atlas by hand. See Section 5.4 for the field-level breakdown of each file.

12.3 Layer 3 — Hardcoded Client Constants

API base URL (`http://localhost:3001/api`, repeated literally across components, no `REACT_APP_*` override), drawer widths (280/72px), mobile breakpoint (768px), theme palette, and the DeepEval endpoint (`http://localhost:8000/eval`) are all literal values requiring a code change to alter.

12.4 Layer 4 — Browser `localStorage`

`darkMode` (theme preference) and `promptSchemaConfig` (Prompt & Schema Manager settings) — neither synced to the server, versioned, or shareable between users.

12.5 What Requires a Restart vs. Takes Effect Immediately

Change	Effect
Edit `.env` directly	Requires manual server restart
Edit via Settings UI	Writes immediately, but **also** requires manual restart
Edit `src/config/*.json`	No effect unless separately re-applied to Atlas
Toggle dark mode / edit Prompt & Schema	Immediate, client-side only

---

13. Security

> **Overall assessment: this application has no meaningful security controls at any layer — network, transport, application, or data.**

13.1 Authentication

None exists. No JWT, session middleware, or per-request identity check anywhere in the 21 routes.

13.2 Authorization

No roles/permissions concept exists. Every caller has full access to every route.

13.3 Secrets Management

`.env` is correctly `.gitignore`-excluded, but that protection is bypassed at the application level: `GET /api/env` returns its full contents (including `MONGODB_URI`, `MISTRAL_API_KEY`, `GROQ_API_KEY`) as plaintext JSON to any unauthenticated caller, and `POST /api/env` allows arbitrary overwrites.

13.4 API Keys

Subject to the same exposure as 13.3. Only a placeholder-string check exists for Groq (`your_groq_api_key_here`) — no equivalent for Mistral.

13.5 Transport Security — MongoDB Connection (confirmed, repo-wide)

Every one of the 10 `MongoClient` instantiations in `server/index.js` explicitly disables TLS certificate and hostname verification:

```
new MongoClient(process.env.MONGODB_URI, {
  ssl: true,
  tlsAllowInvalidCertificates: true,   // disables certificate chain validation
  tlsAllowInvalidHostnames: true,      // disables hostname verification
```

```
  ...
});
```

This is the single most consequential finding in this review — it permits a TLS connection to MongoDB even if the certificate is invalid or presented by a different host, weakening protection against man-in-the-middle interception on every DB connection in the app. No comment in the code explains why this was set; removing these two options should be safe for a correctly configured Atlas SRV connection string.

The Express server itself serves plain HTTP only — no TLS termination of its own.

13.6 Input Validation & Sanitization

No validation library is used anywhere. `POST /api/upload-excel` builds a JavaScript source string by interpolating `req.body.sheetName` and file paths directly into a template literal, writes it to disk, and executes it via `spawn('node', ...)` — a code-injection-shaped pattern (not confirmed exploitable in testing, but a design smell worth replacing with a fixed script taking data via `process.argv`/stdin).

13.7 File Upload Security

`multer({ storage })` — confirmed **no `limits` (size cap) and no `fileFilter` (type allow-list)**. Any file of any type/size can be uploaded, subject only to disk space.

13.8 CORS

`app.use(cors())` with zero configuration — fully permissive default, reflecting any origin.

13.9 Destructive Utilities

`src/scripts/utilities/delete-all-documents.js` runs `collection.deleteMany({})` unconditionally, no confirmation prompt, no environment guard.

13.10 AI-Layer Security

Retrieved content is concatenated into LLM prompts unsanitized; no per-session cost/token ceiling on generation calls; combined with no authentication, any caller can trigger unlimited Groq calls against your API key.

13.11 Dependency Surface

Several unused dependencies (`mysql2`, `sequelize`, `openai`, `@mistralai/mistralai`, `@huggingface/inference`) increase attack surface with no functional benefit. No dependency audit process (`npm audit`) runs anywhere.

13.12 Summary Table

Control	Status
Authentication	✖ None
Authorization / RBAC	✖ None
Secrets exposure via API	✖ `.env` fully readable/writable, unauthenticated
MongoDB TLS validation	✖ Explicitly disabled on every connection
HTTPS (Express)	✖ Plain HTTP only
Input validation library	✖ None
File upload restrictions	✖ No size cap, no type allow-list
CORS restriction	✖ Fully permissive default
Rate limiting	✖ None
Security headers (helmet)	✖ None
Dependency audit / CI	✖ None

13.13 Recommendations (not implemented)

1. **Highest priority:** remove `tlsAllowInvalidCertificates` / `tlsAllowInvalidHostnames` from all 10 `MongoClient` constructions.
2. Add authentication before this is reachable outside `localhost`; reconsider whether `/api/env` should exist at all.
3. Add `multer` `limits` / `fileFilter`.
4. Replace string-interpolated temp scripts with fixed scripts taking data via `process.argv`/stdin.
5. Configure `cors()` with an explicit allowed-origins list before any real deployment.
6. Add `helmet` and `express-rate-limit` if exposed on a shared network.
7. Add a confirmation guard to `delete-all-documents.js`.

8. Add token/cost ceilings around Groq calls and sanitize retrieved content before prompting.

---

14. Performance

14.1 No Connection Pooling Across Requests (confirmed, high-impact)

Every DB-backed route creates a brand-new `MongoClient`, connects, does its work, and closes — within that single request (10 `new MongoClient` sites, 19 corresponding `.close()` calls across success/error/validation branches). This pays a full DNS+TCP+TLS handshake cost on every database-touching API call rather than reusing a single pooled client for the app's lifetime — the single most impactful, mechanically fixable performance issue found in this review.

14.2 Query-Level Performance

- `GET /api/testcases/latest-id` performs a **full collection scan** on every call (Section 8.7/10.10) — no supporting index, invoked on every Prompt & Schema Manager run.
- No standard MongoDB secondary indexes are declared anywhere beyond the named Atlas Search/Vector Search indexes.
- Hybrid Search's BM25 and vector branches run as **sequential awaits**, not concurrently.
- No caching of repeated identical query embeddings — every search regenerates a fresh Mistral embedding.
- No pagination beyond a flat `limit` parameter on any search route.

14.3 Concurrency Controls

Where	Mechanism	Assessment
Batch embedding generation	`p-limit` caps at 3 concurrent Mistral calls; Mongo inserts batched at 100	Well-tuned — a genuine strength
Async job tracking	In-memory `Map`, single process	Fine for one instance; doesn't scale horizontally
Excel conversion / legacy embeddings	Spawns a child `node` process per request	Fine at low volume; wouldn't scale to many concurrent uploads

14.4 Caching

No caching layer exists anywhere — no Redis, no in-memory cache, no HTTP `Cache-Control` / `ETag` headers, no response compression middleware.

14.5 Scalability Ceiling

In-memory job tracking and per-request DB connections both cap this app at a single-instance ceiling — adequate for the demo/single-user use case it appears designed for, but a real constraint before pointing it at a shared team workload.

14.6 Client-Side Performance

MUI `DataGrid` provides its own default client-side pagination; no custom result caching between page navigations was found.

---

15. Logging & Monitoring

15.1 Server-Side Logging

`console.log` / `warn` / `error` only (~110 call sites in `server/index.js`, emoji-prefixed). No structured logging library, no log levels, no timestamps/request IDs attached by the app itself, no log rotation/shipping.

15.2 Error Handling Pattern

21 independent try/catch blocks, each logging via `console.error` and responding with a consistent `{error, details}` shape — but no centralized Express error middleware and no process-level `uncaughtException` / `unhandledRejection` handlers (a real gap given the `setInterval` job-cleanup callback at `server/index.js:80-87`).

15.3 Client-Side Logging

Also `console.*`-only; `EmbeddingsStore.js` includes a visible in-UI "Debug Info" panel left in from development.

15.4 Observability Gaps

No structured logging, request tracing, centralized error handling, crash safety net, APM/tracing, metrics endpoint, or alerting exists. `GET /api/health` is a shallow liveness stub only.

15.5 What Does Exist

- **Per-response cost/token reporting** on nearly every AI-calling route — real-time AI spend visibility per operation, even though not aggregated/persisted.
- **Async job status tracking** (`/api/jobs/*`) — live progress visibility for embedding batches, in-memory only.
- **Search timing breakdowns** (`timing: {...}`) returned inline by hybrid/rerank routes.

15.6 Recommendations (not implemented)

1. Adopt structured logging (e.g. `pino`) with request correlation IDs.
2. Add centralized error-handling middleware as a fallback alongside existing per-route handling.
3. Add `uncaughtException` / `unhandledRejection` handlers.
4. Deepen `/api/health` to optionally verify downstream connectivity.
5. Add basic APM (e.g. a free-tier Sentry project) before anything more elaborate, if deployed beyond one machine.
6. Persist/aggregate the existing per-response cost/token data over time.

---

16. Testing

16.1 Current State

Zero automated tests exist anywhere in this repository's own source. No `test` script at the root; `client`'s CRA test scaffold (`App.test.js`, `setupTests.js`) is absent despite `@testing-library/*` being listed dependencies.

16.2 Unit Testing

None exists. Best-suited targets for a first pass: `src/scripts/query-preprocessing/*.js` (pure functions, no I/O) and the JSON-repair heuristics in `groqClient.js` / `/api/test-prompt`.

16.3 Integration Testing

None exists. No dependency-injection seam around `MongoClient`/Mistral/Groq clients makes mocking harder than typical; Atlas Search/Vector Search specifically aren't available in local Mongo test doubles like `mongodb-memory-server`.

16.4 API / End-to-End Testing

None automated — only the manual curl examples in Section 8 and manual UI clicking exist today. No Postman/`supertest`/Playwright/Cypress present.

16.5 Performance Testing

None — no `k6` / `artillery` / `autocannon` config found. All Section 14 findings are static-analysis-derived, not measured.

16.6 Accessibility Testing

None automated (no `jest-axe` / `axe-core` / `eslint-plugin-jsx-a11y`). MUI provides reasonable accessibility defaults, unverified for this specific app's usage.

16.7 Security Testing

None automated (no SAST/DAST config, no `npm audit` step). Section 13's findings came from manual static review, not an automated scanning pipeline or exploit verification.

16.8 Recommendations, Prioritized

1. Unit tests for `query-preprocessing/*` (fastest path to non-zero coverage).
2. Unit tests for JSON-repair heuristics.
3. `supertest`-based tests for routes with no external dependency (`/api/health`, `/api/files`, `/api/search/deduplicate`).
4. Refactor for testability (inject `MongoClient`/AI clients) before broader integration testing.
5. A minimal CI workflow running the above plus `npm audit`.

---

17. Deployment

17.1 Current State

No CI/CD, no Docker, no IaC, no cloud hosting config anywhere in this repository. **No git history exists at all** — confirmed via `git log` / `git tag` / `git branch`: zero commits, zero tags, zero branches. No `LICENSE` or `CHANGELOG` file at the project root.

17.2 Clarifying `releases/`

`releases/1.21.2/stories/` is sample demo data (three healthcare-domain `.txt` user stories), not a release-artifacts mechanism — it has no relationship to `package.json`'s `"version": "1.0.0"` (the numbers don't even match).

17.3 Manual Path to Running This "In Production" Today

1. `npm run build` the client.
2. Serve `client/build/` with a static host of your choice — the API server doesn't do this.
3. Fix the hardcoded client API base URL before the built client can reach a non-localhost API.
4. Run `node server/index.js` under a process manager (`pm2` / `systemd`) — nothing here restarts it automatically.
5. Provide production `.env` — address Section 13's findings first, given `/api/env`'s exposure.
6. Point `MONGODB_URI` at production Atlas with indexes already created (Section 5.4).
7. Add a reverse proxy/TLS termination if exposed beyond a private network.

17.4 Illustrative Containerization (recommended, not present)

A minimal setup would typically include a `Dockerfile` for the API (`node:20-slim`, `npm ci`, `node server/index.js`) and a separate static-hosting stage/image for `client/build/`, wired together via `docker-compose.yml` with `.env` injected rather than baked in. **None of this exists in the repo today** — it's a suggestion for future work.

17.5 CI/CD (recommended, not present)

A minimal pipeline would run install, lint, test, and build on every push/PR. None exists today.

17.6 Release Process (recommended, not present)

With zero git history and no `CHANGELOG`, there is no way to answer "what changed between versions" today. Recommended: initialize git history, adopt semantic versioning aligned with `package.json`, tag releases, and maintain a `CHANGELOG.md`.

---

18. Troubleshooting

18.1 Setup & Startup

Symptom	Root Cause	Solution
`npm install` fails to set up client deps	`postinstall` run from the wrong working directory	Run `npm install` from the project root, or `cd client && npm install` manually
Every DB route returns 500 immediately	`MONGODB_URI` missing/malformed, or Atlas unreachable (IP allow-list)	Verify `.env`; confirm your IP is allow-listed in Atlas Network Access
`/api/health` is OK but search/embeddings still fail	Health check doesn't verify downstream connectivity	Check the specific route's `details` field instead

18.2 Database, Collections & Search Indexes

Symptom	Root Cause	Solution
`"Database '<name>' not found"`	`DB_NAME` mismatch, or missing `listDatabases` privilege	Verify spelling; confirm the database exists
`"Collection '<name>' not found..."`	Collection doesn't exist yet	Run an embedding job once, or create manually
`"No documents found in collection..."`	Search attempted before embeddings were created	Run Convert → Embed first
`"Search index '<name>' not found..."`	Index name in `.env` doesn't match an Atlas index	Create the index per Section 5.4
User-story search always shows `"vector-only"`	`USER_STORIES_BM25_INDEX_NAME` unset (undocumented var)	Add it to `.env` by hand and restart
Vector search returns nothing despite data existing	Index `numDimensions` mismatch, or index still building	Confirm 1024 dims matches `src/config/testcases-vector-index.json`; check Atlas index build status

18.3 Uploads & Conversion

Symptom	Root Cause	Solution
`"No file uploaded"`	Form field not named `file`	Match the field name exactly
`"No sheets found"` / `"No data found in sheet..."`	Empty workbook or wrong sheet name	Confirm sheet has data; leave `sheetName` blank to use the type default
`"Conversion script timeout after 30 seconds"`	Very large Excel file	Split into smaller files — timeout isn't configurable today

18.4 Embeddings

Symptom	Root Cause	Solution
`"No files selected"`	Nothing checked in the file grid	Select at least one file
`"Batch script not found..."`	Invalid `scriptName`	Only the two known scripts exist; shouldn't occur via normal UI use
Job stuck at `"in-progress"` indefinitely	Crashed batch process, or long-running job	Check server console logs; job state is lost on restart
`404 Job not found` for a recent job	Job pruned after 1hr, or server restarted (in-memory store)	Start a new job

18.5 AI / LLM

Symptom	Root Cause	Solution
`"GROQ API KEY is required..."`	Key unset or still the placeholder string	Set a real key from console.groq.com, restart
Mistral routes fail with an unclear error	Invalid Mistral key — no placeholder check exists for it	Double-check the key value directly
Generated test cases look truncated	Possible `maxTokens` mismatch between client (10,000) and server cap (8,000) — flagged, not fully confirmed	Try a smaller schema/target output length
Reranking silently returns unranked order	`rerankDocuments()` falls back to original order on any Groq failure, by design	Check server logs for the underlying Groq call failure

18.6 UI-Specific

Symptom	Root Cause	Solution
"Before"/"After Reranking" tabs look identical	Confirmed limitation — API doesn't return a distinct pre-rerank list	No client-side fix available today
"Metrics Evaluation" tab errors immediately	Calls an external service on port 8000 not included in this repo	Stand up a compatible service, or ignore this tab
Settings says "saved" but behavior doesn't change	`.env` changes never hot-reload	Restart `node server/index.js`
Network/CORS errors from a non-local machine	API base URL hardcoded to `localhost:3001`	Requires a source change to point at the real API host

18.7 Data Loss

Symptom	Root Cause	Solution
A collection is unexpectedly empty	`delete-all-documents.js` was run — no confirmation, no undo	Re-run the ingestion pipeline from source data; be careful which `.env` is active before running this script

---

19. FAQ

1. What is this project? A reference RAG pipeline for searching/generating QA test cases from user stories, using MongoDB Atlas, Mistral, and Groq. See Section 1.

2. Who is this for? QA engineers evaluating retrieval strategies, and developers/architects wanting a working RAG reference to adapt.

3. Is this production-ready? No — no authentication, no automated tests, no CI/CD, and confirmed security gaps (Sections 13, 16, 17).

4. Is this a healthcare product? No — the app is domain-agnostic; the bundled sample data and some prompts are healthcare-themed demo content only.

5. Is there a CLI, or UI-only? Both — most pipeline stages also have standalone CLI scripts in `src/scripts/`.

6. What do I need before running this? Atlas with Vector Search + Atlas Search enabled, a Mistral key, a Groq key. See Section 5.1.

7. Do I need to manually create search indexes? Yes — the app only checks index names exist; it never creates or applies them.

8. Why two AI providers? Mistral for embeddings only, Groq for all generative tasks — an implementation choice, not a functional requirement.

9. Can I run this with Docker? Not out of the box — no Dockerfile exists.

10. Does `npm install` set up the frontend too? Yes, via the root `postinstall` script.

11. What file formats can I upload? Excel (`.xlsx`) only, for test cases or user stories.

12. What happens to my uploaded Excel file afterward? It's converted to JSON and then deleted from `uploads/`.

13. Why is "create embeddings" separate from "convert to JSON"? Conversion just reshapes data; embedding is a separate, costed Mistral API call you may want to review before running.

14. What's the difference between the two embedding endpoints? `/api/create-embeddings` is per-document/legacy; `/api/create-embeddings-batch` batches for speed and is what the UI uses.

15. If I re-run embeddings on the same file, does it overwrite old documents? No — both paths insert new documents; re-running creates duplicates.

16. Do generated test cases get saved back into the database automatically? No — only CSV export; re-ingestion requires manual re-upload through the normal pipeline.

17. What's the difference between Vector, BM25, Hybrid, and Score Fusion search? Vector = semantic; BM25 = keyword; Hybrid = weighted numeric combination; Score Fusion = LLM reranking (not a numeric fusion, despite the name).

18. Which one should I use? BM25 for exact IDs/keywords, Vector for concept matches, Hybrid to balance both, Score Fusion for LLM judgment at higher latency/cost.

19. Why do BM25 and vector scores look so different in scale? BM25 is unbounded; vector is 0–1 cosine similarity. Hybrid Search normalizes both before combining.

20. Can I search user stories the same way? Yes, via `/api/search/user-stories`, which also adds `impactReason` / `regressionRisk`.

21. Why did my user-story search run vector-only? `USER_STORIES_BM25_INDEX_NAME` isn't set — an undocumented env var.

22. Which specific models are used by default? `mistral-embed` for embeddings; Groq models configurable via env vars (see Section 3.4 for exact defaults/fallbacks).

23. Is reranking deterministic? No — it's an LLM call; identical input can yield different ordering across runs.

24. What does the Prompt & Schema Manager do? Runs the full pipeline automatically end-to-end: preprocess → search → rerank → dedup → summarize → generate new test cases.

25. Where is my custom prompt/schema saved? Only in that browser's `localStorage` — not shared across browsers or users.

26. Why does "Metrics Evaluation" fail? It calls an external service on port 8000 not included in this repository.

27. Does reranking/summarization cost money? The app reports Groq cost as always `$0` (treated as free-tier in code); Mistral embedding costs are estimated and reported per call.

28. Is it safe to expose this on the internet as-is? No — no authentication on any route, and `/api/env` exposes/allows overwriting all secrets.

29. Who can see my API keys once the server is running? Anyone who can reach the server's port, via unauthenticated `GET /api/env`.

30. Does this app have user accounts/login? No.

31. How many concurrent users can this handle? Designed for single-developer/local use — per-request DB connections (no pooling) and in-memory job tracking both limit concurrent/multi-instance scaling.

32. Does the app cache search results or embeddings? No caching layer exists anywhere.

33. Are there automated tests? No — zero test files exist despite testing libraries being listed as dependencies.

34. Is there a CI pipeline? No CI/CD configuration exists.

35. Can I point this at a different domain than QA test cases? Architecturally yes — the pipeline stages are domain-agnostic; you'd need to adapt Excel column mappings, embedding input templates, and prompt/schema content.

36. Can I swap Mistral or Groq for another provider? Not via configuration — the provider wrapper modules are called directly with no abstraction layer; swapping requires editing their internals.

37. What's in `src/scripts/search/`, and can I use it? A legacy, orphaned implementation using a different embedding provider — not called by the live app and would need adaptation before reuse.

---

20. Future Enhancements

Every item below is a **recommendation**, not a description of anything implemented today.

20.1 Critical — Security & Data Safety

- Add authentication/authorization middleware to all routes.
- Remove or gate `GET` / `POST /api/env` behind auth.
- Remove `tlsAllowInvalidCertificates` / `tlsAllowInvalidHostnames` from all `MongoClient` constructions.
- Add `multer` `limits` / `fileFilter`.
- Add a confirmation guard to `delete-all-documents.js`.

20.2 High — Correctness & Consistency

- Implement true numeric fusion (e.g. RRF) for "Score Fusion," or rename the UI to reflect actual LLM-reranking behavior.
- Return a genuine pre-rerank candidate ordering from `/api/search/rerank`.
- Reconcile the `maxTokens` cap mismatch between client and server.
- Normalize `embeddingMetadata.apiSource` labeling across both embedding paths.
- Document `USER_STORIES_BM25_INDEX_NAME` / `PORT` in `.env.example` and the Settings UI.
- Add upsert-by-ID (or duplicate detection) to the embedding pipeline.

20.3 Medium — Performance & Scalability

- Instantiate one `MongoClient` at startup, reused across all routes.
- Back `/api/testcases/latest-id` with an index or counter document.
- Move job tracking to a shared store (Redis, or persisted to Mongo).
- Add a query→embedding cache for repeated searches.
- Dispatch Hybrid Search's two branches concurrently.

20.4 Medium — Observability & Quality

- Structured logging with request correlation IDs.
- Centralized error middleware + process-level crash handlers.
- An initial automated test suite, starting with pure logic modules.
- A minimal CI pipeline (install, lint, test, build, `npm audit`).

20.5 Nice-to-Have — Feature & Developer Experience

- Server-side persistence for Prompt & Schema configuration.
- Env-driven client API base URL instead of hardcoded `localhost`.
- Serve `client/build/` from the Express server, or document a concrete static-hosting recipe.
- Cursor/`skip`-based pagination on search routes.
- Bundle or clearly document the external metrics-evaluation service dependency.
- A Dockerfile/`docker-compose.yml` for one-command local spin-up.
- Remove or genuinely wire up unused dependencies (`mysql2`, `sequelize`, `openai`, `@mistralai/mistralai`, `@huggingface/inference`, `react-router-dom`).

20.6 What's Already Solid (worth preserving)

- The side-by-side multi-strategy search UX (Vector/BM25/Hybrid/Rerank as comparable pages).
- The batch embedding pipeline's concurrency tuning (`p-limit`, retry/backoff).
- Graceful LLM-rerank fallback on failure.
- Per-response cost/token/timing visibility built into API responses.

---

21. Contributing

21.1 Current State

No contributing guidelines exist in this repository. No `CONTRIBUTING.md`, no root-level lint/format config, no PR/issue templates, and — confirmed via `git log` / `git tag` / `git branch` — **zero commits, branches, or tags exist**, so there is no established convention to document. Everything below is a **proposed baseline**.

21.2 Recommended Coding Standards

- Adopt a root-level ESLint config for `server/` / `src/scripts/` (currently only `client/` has any linting, via CRA defaults).
- Add Prettier and a pre-commit hook.
- Continue using ES modules consistently, matching the existing codebase.
- As the codebase grows, consider splitting `server/index.js`'s 21 routes into separate router files.
- Never commit `.env` — already correctly `.gitignore`-excluded; keep it that way as new variables are added.

21.3 Recommended Branching Strategy

Trunk-based: `main` as the primary branch, short-lived `feature/` / `fix/` branches merged via pull request. Given the absence of tests/CI, manual verification against Section 5.6's checklist should be treated as a mandatory PR step until real CI exists.

21.4 Recommended Commit Conventions

Conventional Commits (`feat:`, `fix:`, `docs:`, `refactor:`, `chore:`) — a reasonable default given no existing convention to preserve, and one that would make a future `CHANGELOG.md` generatable from history.

21.5 Recommended Pull Request Process

1. Branch from `main`, make focused changes.
2. Manually verify against Section 5.6 before opening a PR — no CI does this today.
3. If touching a route in `server/index.js`, update the corresponding entry in Section 8 of this document.
4. Flag any change touching auth, `.env` handling, or MongoDB connection options explicitly in the PR description, given Section 13's findings (e.g. the disabled TLS validation is easy to reintroduce accidentally if copy-pasted).
5. Describe manual test steps taken in the PR description until Section 16's testing recommendations are adopted.

21.6 Getting Started as a Contributor

Read, in order: Section 1 (what this is), Section 2 (architecture), Section 7 (where things live — especially the Active-vs-Orphaned distinction for `src/scripts/`), and Sections 5–6 (how to run it) before making changes.

---

22. License

No license is declared anywhere in this repository. Confirmed: there is no `LICENSE` / `LICENSE.md` file at the project root, and neither `package.json` (root or `client/`) includes a `"license"` field.

Under default copyright law, the **absence of a license file does not mean this code is free to use, modify, or redistribute** — all rights are reserved by the author(s) by default unless and until an explicit license is added. Anyone intending to share, distribute, or accept external contributions to this project should add an explicit `LICENSE` file (e.g. MIT, Apache 2.0, or a proprietary/internal-use license, depending on intent) and a matching `"license"` field in both `package.json` files — this document does not select one on the project owner's behalf, since that is a legal/business decision outside the scope of a code-level audit.

This document reflects a full code-level review of the repository as of the time it was written. Sections referencing "confirmed," "verified," or specific file/line numbers were checked directly against the source; items marked "unable to determine" or "recommended" were explicitly not present in the code and should not be treated as implemented.